

SQL Injection



SQL Injection

- L'SQL Injection è un particolare tipo di attacco il cui scopo è quello di indurre il database ad eseguire query SQL non autorizzate.



Esempio: login

Nome Utente :

Password :



```
$utente=$_POST["nUtente"];
$pass=$_POST["pass"];

$con = new PDO("mysql:host=$server;dbname=$dbname", $user, $psw);
$con->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

$req = "select * from utenti where nomeUtente='".$utente."'and password='".$pass."'";

$statement = $con->prepare($req);
$statement->execute();
$res = $statement->fetchAll();
```

Esempio: login

Nome Utente :

Password :

← password corretta

Accedi

```
$req = "select * from utenti where nomeUtente='".$utente."'and password='".$pass."';";
```



```
$req = "select * from utenti where nomeUtente='admin'and password='123';";
```



Bentornato Andrea

Esempio: login

Nome Utente :

Password :

← password errata

Accedi

```
$req = "select * from utenti where nomeUtente='".$utente."'and password='".$pass."';";
```



```
$req = "select * from utenti where nomeUtente='admin'and password='1';";
```



Login fallito

Attacco SQL Injection (1)

Nome Utente :

Password :

← password errata

Accedi

```
$req = "select * from utenti where nomeUtente='".$utente."'and password='".$pass."';";
```



```
$req = "select * from utenti where nomeUtente='admin' OR '1'='1' and password='1';";
```

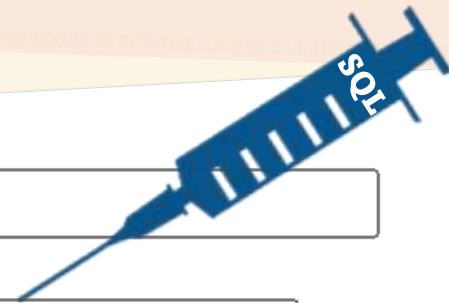


Bentornato Andrea

Attacco SQL Injection (2)

Nome Utente :

Password :



← ' OR '1'='1' --

```
$req = "select * from utenti where nomeUtente='" . $utente . "'and password='" . $pass . "';";
```



```
$req = "select * from utenti where nomeUtente='admin'and password='' OR '1'='1' --';";
```



Bentornato Andrea

Attacco SQL Injection (3)

1; DROP TABLE users;



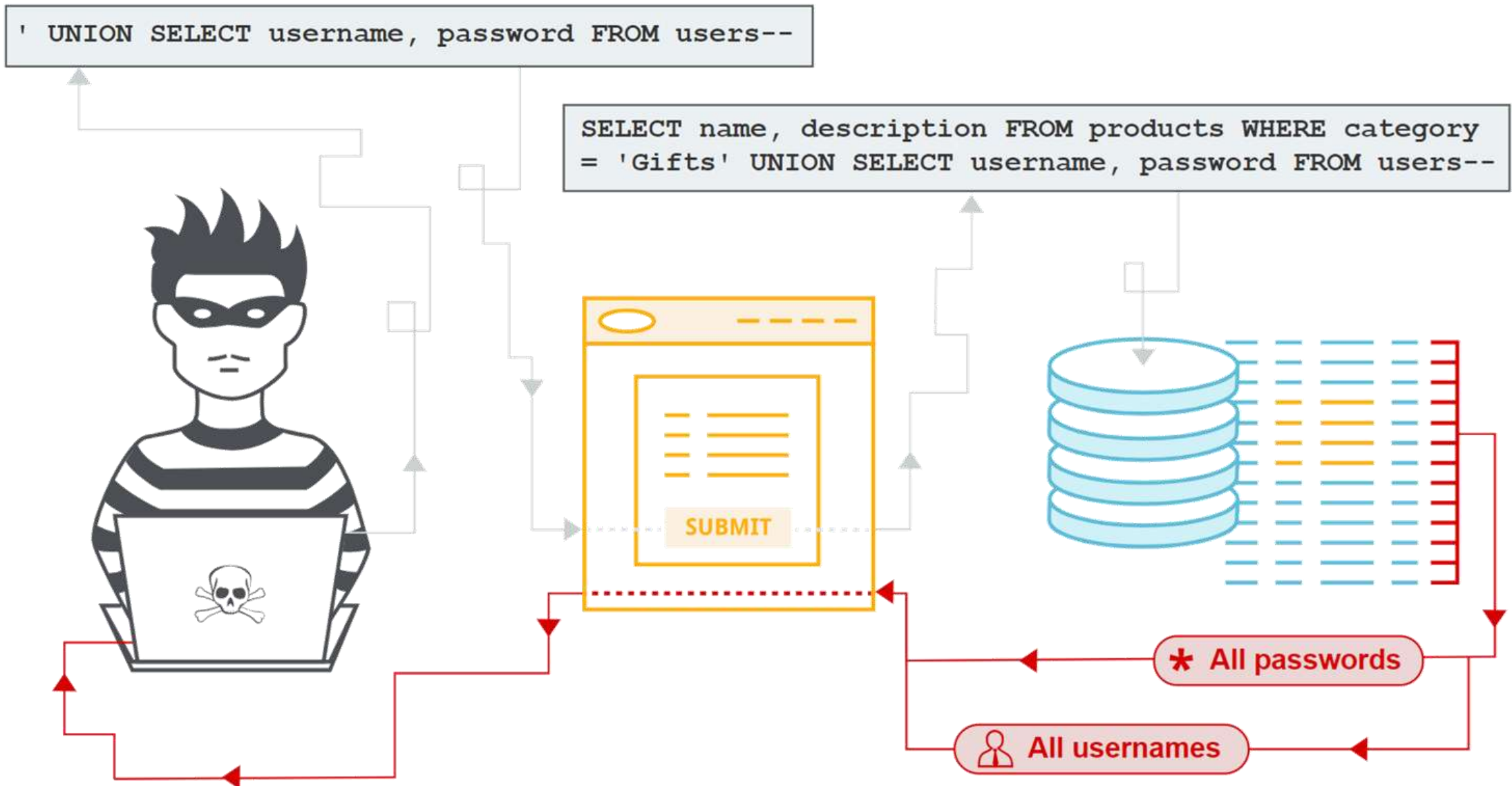
SELECT * FROM users WHERE id = 1; DROP TABLE users;

SELECT * FROM users WHERE id = \$id;

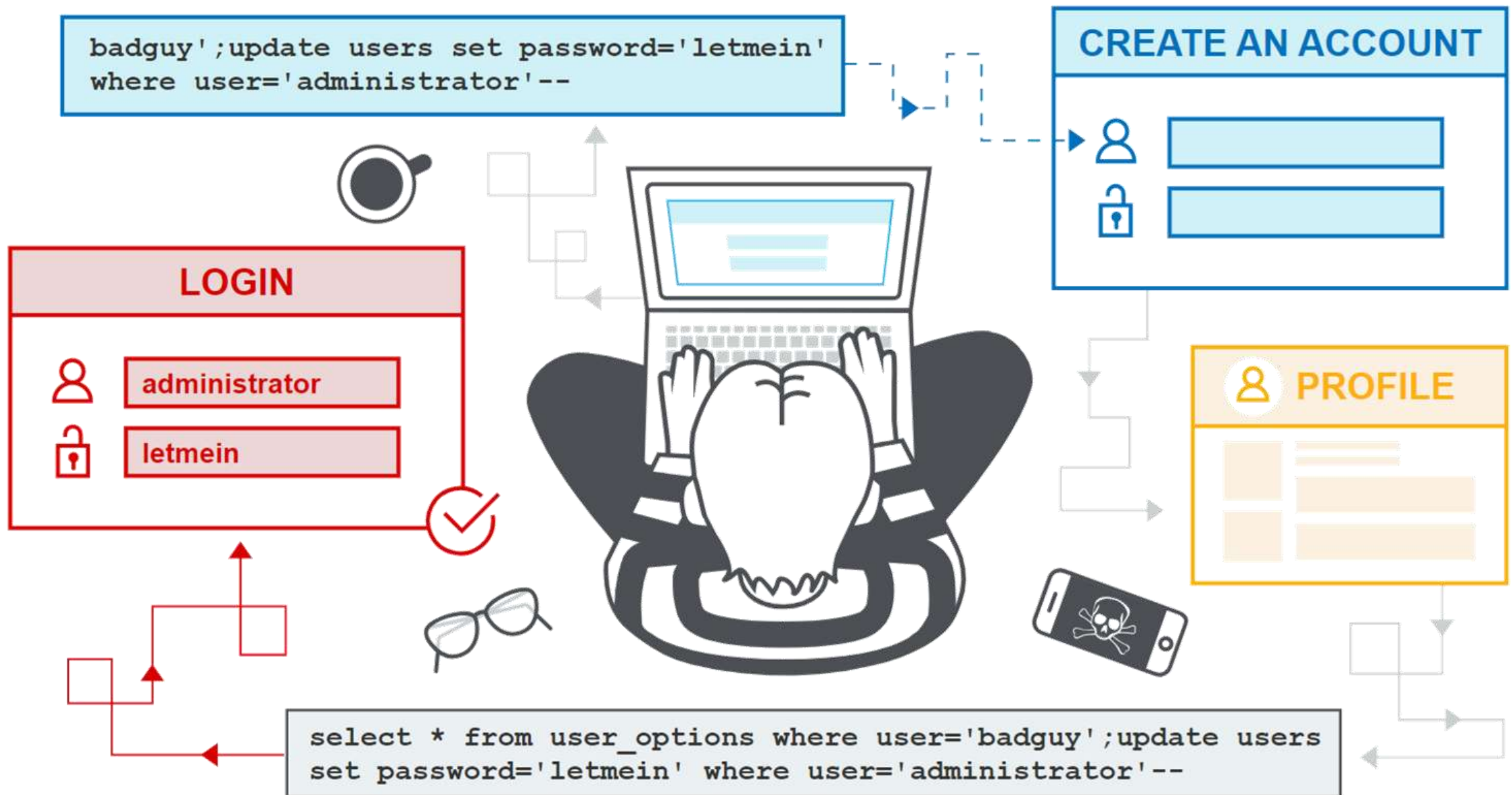
- Se il server è configurato in modo non sicuro, potrebbe eseguire entrambe le istruzioni, causando la cancellazione della tabella degli utenti.



Attacco SQL Injection (4)



Attacco SQL Injection (5)



Attacco SQL Injection (6)



➤ Time-based SQL Injection

➤ Query originale

```
SELECT name, description FROM products WHERE category = '$category';
```

➤ Injection

```
' AND (SELECT IF(SUBSTRING(user, 1, 1) = 'a', SLEEP(10), 0) FROM users  
WHERE id = 1) --
```

➤ Query risultante

```
SELECT name, description FROM products WHERE category = '' AND  
(SELECT IF(SUBSTRING(user, 1, 1) = 'a', SLEEP(10), 0) FROM users WHERE id = 1) --';
```



Sfide di SQL Injection

➤ <https://redtiger.labs.overthewire.org/>



RedTiger's Hackit

Welcome to my 1st hackit. Visitors: 289287.

This hackit is for people who want to test their knowledge in PHP / SQL security.

It has some similarities to h0yt3r's and shadowleer's sql-injection hackits but it will also test you in some logical ways of thinking.

All levels are based on real vulnerabilities I found in the wild.

Be honest. Dont bruteforce the passwords and **dont make any solutions public!!!**

Well I am not a friend of long speeches so just start now.

Special challenge

You can see my contact information when level2 is solved

---> [Start](#) <--- here

Notice: Level4 is the only level you need to exploit blindly to get the value

I know that my anti-blind-checks are not very consistent. So find the right way to exploit the levels. Anyway, have fun!

Start here -->	Level 1	Simple SQL-Injection	solved by 21826 hackers
	Level 2	Simple login-bypass	solved by 15645 hackers
	Level 3	Get an error	solved by 4843 hackers
	Level 4	Blind Injection	solved by 2838 hackers
	Level 5	Advanced login-bypass	solved by 2331 hackers
	Level 6	SQL-Injection	solved by 1549 hackers
	Level 7	SQL-Injection	solved by 1202 hackers
	Level 8	SQL-Injection	solved by 1088 hackers
	Level 9	SQL-Injection	solved by 987 hackers
	Level 10		solved by 873 hackers

➤ <https://portswigger.net/web-security/all-labs>

Come evitare SQL Injection?

- Assegnare permessi di accesso più restrittivi
- Inserendo nelle pagine PHP dei meccanismi di validazione dei dati forniti dall'utente
 - Controlli sui limiti, tipo di dato, ...
- Separando il codice dai dati
 - Prepared statement



Prepared statement

- Le prepared statement sono utilizzate per migliorare le prestazioni e la sicurezza delle query SQL.
 - Aiutano a prevenire attacchi di tipo SQL Injection.
- Si tratta di query SQL precompilate, ottimizzate e memorizzate nel database.

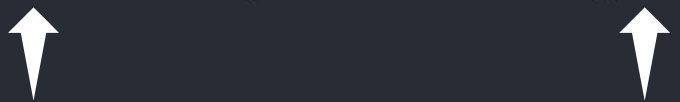
```
$statement = $con->prepare($req);
```

Prepared statement e parametri nominali



- I parametri nominali sono dei **segnaposto** nominali per i valori che saranno successivamente passati al comando.
 - Si tratta di nomi significativi preceduti dai due punti (:user, :psw)

```
$req = "select * from utenti where nomeUtente = :user and password = :psw;";  
$statement = $con->prepare($req);
```



- Il metodo *prepare* predispone il modello di query

- Il metodo *bindParam* associa un parametro nominale (1° parametro) ad una variabile (2° parametro) e al tipo della variabile stessa (3° parametro)

```
$req = "select * from utenti where nomeUtente = :user and password = :psw;";  
$statement = $con->prepare($req);  
  
$statement->bindParam(':user', $utente, PDO::PARAM_STR);  
$statement->bindParam(':psw', $pass, PDO::PARAM_STR);
```

- Si possono specificare 3 tipi di dato: PDO::PARAM_BOOL, PDO::PARAM_INT e PDO::PARAM_STR.



- Il metodo *execute* esegue la query.
- La stessa prepared statement può essere eseguita più volte con valori diversi per i parametri.

```
$req = "select * from utenti where nomeUtente = :user and password = :psw;";  
$statement = $con->prepare($req);  
  
$statement->bindParam(':user', $utente, PDO::PARAM_STR);  
$statement->bindParam(':psw', $pass, PDO::PARAM_STR);  
  
$statement->execute();  
$res = $statement->fetchAll();
```

Prepared statement e parametri nominali

- L'utilizzo di prepared statement costituisce una delle pratiche più efficaci per prevenire l'SQL Injection in PHP.
- Anche se un attaccante inserisce codice SQL dannoso, verrà trattato come un dato e non eseguito come parte della query.

